

Claude Code 项目工程工作流手册

从任务边界、上下文治理、规格化、实现闭环，到验证门禁、交付审查与团队化沉淀的工程协议。

手册定位

Claude Code 不是单轮问答工具，而是能够读取代码库、编辑文件、运行命令并接入外部工具的 agentic coding system。工程流程的重点因此从“表达愿望”转向“建立可执行、可验证、可审查的闭环”。

本文不是官方单一 SOP。它基于 Claude Code 官方文档与 Anthropic 工程博客，重组为面向项目交付和团队治理的工作流手册。

Winston

资料口径截至 2026-05-04

官方文档 · 工程博客 · 项目实践模板

00

目录与阅读路径

从概念模型进入
以交付门禁结束

01	读者契约	边界与方法	08	验证门禁	tests · review · evidence
02	核心模型	context · action · verification	09	交付协议	commit · PR · CI · release
03	项目流程总图	七阶段 workflow	10	扩展机制	MCP · hooks · skills · subagents
04	准备层	Git · CLAUDE.md · commands	11	团队治理	roles · policies · maturity
05	上下文治理	memory · rules · context	12	失败模式	风险信号与纠偏
06	规格与计划	SPEC · acceptance · plan mode	13	模板与来源	可复用资产
07	实现闭环	vertical slice · delegation			

READING CONTRACT

本手册适合三类读者：需要把 Claude Code 纳入研发流程的技术负责人；希望减少返工和上下文失控的工程师；准备把个人使用经验沉淀为团队标准的项目维护者。正文不讨论通用编程规范，也不把模型能力作为结论来源；所有流程建议均落在项目交付、验证和治理层面。

01 | 读者契约：从聊天技巧转向工程协议

目标不是“让工具更听话”
而是让交付可控

Claude Code 的官方定位决定了流程设计的起点：它能够读取代码库、编辑文件、运行命令、使用 Git，并通过 MCP、hooks、skills、subagents 等机制扩展能力。项目 workflows 的核心不应是单句任务指令，而应是可重复的工程协议。

PRINCIPLE 01

边界先行

任务开始前明确工作树状态、可改范围、非目标、权限边界和验收证据。边界不清时，后续纠偏成本通常高于前置澄清。

PRINCIPLE 02

上下文可治理

长期规则进入 `CLAUDE.md` 或 `scoped rules`；临时发现进入 `checklist`；调查工作交给隔离上下文。对话历史不应成为唯一记忆层。

PRINCIPLE 03

规格化优先

复杂需求先形成规格、验收标准和实施计划。计划不是形式，而是把目标、风险、验证路径和代码库现实对齐。

PRINCIPLE 04

证据交付

完成状态由测试、类型检查、构建、界面快照、人工路径或 CI 证明。没有证据的“完成”不进入交付。

适用边界

适合	不适合	处理方式
功能开发、缺陷修复、重构、迁移、文档同步、PR review、CI 分析。	未授权生产操作、无回滚数据库变更、机密数据外发、无审查权限变更。	将高风险动作从自动执行链路中剥离，保留人工审批和可审计记录。
已有代码库或明确项目目标。	目标含混、无验收标准、无法运行项目、无法确认依赖来源。	先进入规格化阶段，补齐项目准备资产。

02 | 核心模型：上下文、行动、验证

Claude Code 的基本循环
也是项目流程的骨架

官方文档将 Claude Code 的工作过程概括为 gather context、take action、verify results。真实项目中，这三者不是一次性瀑布，而是围绕工具反馈持续收敛的循环。工程流程的价值，是让每一次循环都有目标、边界和退出标准。

01

Gather Context

读取代码、Git 状态、文档、命令输出、错误日志、设计材料和项目规则。此阶段的关键不是读取越多越好，而是限定问题域、识别相关文件和已有模式。

02

Take Action

编辑文件、运行命令、启动服务、修改配置、生成文档或创建提交。行动必须位于授权范围内，并尽量采用最小可审查变更。

03

Verify Results

运行测试、lint、typecheck、build、人工路径或视觉对照。失败不是异常，而是下一轮行动的输入。

CONTROL

权限模式、allowlist、sandbox、分支、工作树和人工审批构成控制面。控制面缺失时，自动化能力会放大事故半径。

MEMORY

CLAUDE.md、auto memory、rules、skills 和 subagents 构成记忆面。记忆面过载会降低遵循度，过少则导致重复解释。

EVIDENCE

测试输出、构建结果、界面快照、审查意见和风险记录构成证据面。证据面决定交付是否可信。

03

项目流程总图：七阶段工作流

每个阶段都有产物
没有产物就不算完成

把 Claude Code 纳入项目工程，不应从“让它开始写代码”开始，而应从建立可回滚、可验证、可审查的工作系统开始。七阶段工作流适用于从小修复到中型功能开发；大型项目可以把每个阶段拆成多个迭代。



阶段	入口条件	退出条件	失败信号
Project Ready	项目可访问，工作区可检查。	命令、规则、权限和回滚边界明确。	测试无法运行，已有改动归属不明。
Task Framing	存在明确任务或问题。	目标、非目标、验收、风险写入任务材料。	只描述功能名，没有成功标准。
Plan	复杂度超过局部小改。	相关文件、方案、验证路径和风险已确认。	未经阅读代码库即开始批量修改。
Build / Verify	计划已被接受。	实现通过证据门禁。	只靠人工观感判断完成。
Ship / Institutionalize	diff 可审查，风险可解释。	PR 信息完整，经验进入长期资产。	相同提醒在多个任务中重复出现。

04 | 准备层：把项目变成可操作环境

准备不是前戏
它直接决定交付质量

Claude Code 能读取 Git 状态、编辑文件、运行命令并创建提交，但这些能力需要项目自身提供边界。准备层的目标，是让每次任务都能回答三个问题：当前可改什么、如何证明正确、哪些动作需要人工审批。

最小项目包

资产	作用	质量标准
CLAUDE.md	项目级规则与命令。	少于 200 行；只写每次会话都需要的事实。
测试命令	验证闭环入口。	区分最小相关测试与完整测试。
Git 分支	隔离改动。	任务分支明确，未提交改动归属清楚。
权限策略	限制自动动作。	低风险命令可 allowlist，高风险动作保留确认。
回滚路径	降低事故成本。	Git、feature flag、迁移回滚或配置回滚可描述。

工作区启动检查

```
git status --short
git branch --show-current
git log --oneline -n 5
[minimal test command]
[lint or typecheck command]
```

ENGINEERING JUDGMENT

Claude Code 的 checkpoints 能撤销文件编辑，但不能替代 Git，也不能覆盖外部副作用。生产部署、数据库迁移、权限变更、外部写入和 force push 不应交给未审查的自动链路。

CLAUDE.md 的写法

- 写项目特有规则，不写通用编程常识。
- 写可执行命令，不写模糊偏好。
- 写模块边界，不复制完整 README。
- 写 compact instructions，避免长任务中丢失关键状态。

05

| 上下文治理：把信息放在正确层级

上下文越多不等于越好
关键是分层与作用域

官方 Memory 文档说明，每个会话从新的 context window 开始，`CLAUDE.md` 与 auto memory 会在启动时加载。上下文治理的目标不是保存一切，而是让长期规则、临时状态、局部约束和一次性调查分层存放。

PERSISTENT

`CLAUDE.md` 与 rules

适合构建命令、项目架构、编码标准、验证规则和安全边界。大项目可用 `.claude/rules/` 做路径作用域。

SESSION

checklist 与 scratchpad

适合迁移清单、失败测试、待处理文件和阶段性决策。任务完成后归档或删除，不进入长期上下文。

ISOLATED

subagents

适合会污染主会话的调查任务，例如日志归纳、代码搜索、依赖审计和独立 review。子任务返回摘要，不把原始噪音带入主线。

信息分层决策表

信息类型	推荐位置	理由	不建议
每个任务都要运行的命令	<code>CLAUDE.md</code>	启动时加载，稳定可见。	反复粘贴到会话。
仅适用于某目录的规则	<code>.claude/rules/</code>	减少无关上下文。	塞进根级长文件。
个人偏好或本地地址	<code>CLAUDE.local.md</code>	不应进入版本控制。	提交到共享仓库。
一次性探索结果	临时 checklist	任务结束后可清理。	进入长期规则。
重复流程	skills	以流程资产复用。	靠个人口头习惯。

06 | 规格与计划：先定义验收，再进入实现

复杂任务不应直接编码
先形成可执行规格

复杂任务失败，通常不是因为实现能力不足，而是目标、边界、验收和现有代码现实没有对齐。规格化阶段应产出两个文件：`SPEC.md` 说明要解决什么、为什么、做到哪里；`ACCEPTANCE.md` 说明如何证明完成。

规格文件的必要字段

字段	判断标准
Problem	描述问题和后果，而不是功能名。
Actors	区分终端用户、管理员、维护者和外部系统。
Scope	同时写 in scope 与 out of scope。
Data / Interface	列出输入、输出、错误状态和迁移要求。
Acceptance	包含测试、人工路径、界面快照或性能阈值。
Risks	说明安全、兼容性、性能和回滚风险。

计划阶段的标准指令

进入只读计划阶段。请先研究与 [目标] 相关的代码，不修改文件。

输出：

- 1. 相关文件与现有模式
 - 2. 推荐实现方案
 - 3. 替代方案与取舍
 - 4. 验证计划
 - 5. 风险与回滚路径
- 计划被确认后再实现。

PLAN MODE

官方文档建议复杂问题先探索再实现。Plan mode 的价值在于将代码库阅读、方案选择和风险识别从文件修改中分离出来，避免一边探索一边制造不可审查 diff。

任务拆分原则

- 先做 skeleton，再做 happy path，再扩展边界。
- 每个 vertical slice 都应能运行、能验证、能回滚。
- 批量迁移先做样本，验证后再扩大范围。

07 | 实现闭环: 委派目标, 不微操路径

Claude Code 擅长跨文件行动
前提是目标和边界明确

官方 How Claude Code works 强调 delegate, don't dictate。工程师应给出目标、约束、上下文和验收方式, 而不是规定每个文件和每一行修改。过度微操会削弱工具根据代码库结构自行选择路径的能力。

角色 / 阶段	调查	计划	实现	验证	交付
项目负责人	给目标、非目标、业务规则和高风险边界。	审查方案与取舍。	只在方向偏离时介入。	确认验收证据是否充分。	决定合并、发布和回滚策略。
Claude Code	搜索、读取、定位相关文件。	提出实施路径、风险和验证计划。	修改文件并保持 diff 聚焦。	运行命令, 依据失败反馈修正。	整理摘要、测试结果和 PR 信息。
Reviewer	不参与初始上下文。	可审查计划假设。	不在实现中途扩大范围。	独立检查行为回归和缺失测试。	审批或要求修正。

高质量任务指令结构

OBJECTIVE

实现或修复的具体目标, 最好包含用户路径或工程影响。

CONTEXT

相关 issue、错误日志、设计材料、路径线索和现有模式。

CONSTRAINTS

不得修改的范围、依赖限制、兼容性要求和安全边界。

VERIFICATION

必须运行的命令、人工路径、界面快照或性能阈值。

HANDOFF

要求输出变更摘要、验证结果、未验证区域和残余风险。

STOP RULE

遇到产品判断、权限动作、不可复现错误或不确定迁移时暂停说明。

08 | 验证门禁：交付必须有证据

完成不是状态声明
完成是证据集合

Claude Code 在有可验证反馈时表现更稳定。测试、构建、类型检查、人工路径和视觉对照不是交付后的附属动作，而是实现循环的一部分。验证越晚，返工越贵。

任务类型与证据

任务类型	最低证据	高风险补充
纯函数 / API	单元测试或契约测试。	边界输入、错误状态、性能样本。
缺陷修复	复现路径与回归测试。	根因说明和相邻风险检查。
UI	目标视口的界面快照和交互路径。	键盘、可访问性、加载与错误状态。
重构	现有测试、类型检查和公共接口说明。	性能基线和兼容性抽样。
迁移	样本迁移、抽样验证和回滚说明。	批量 checklist 与 CI 追踪。

Evidence Ledger

Verification:

- Command: [command]
Result: [passed / failed / not run]
Notes: [key output]
- Manual path: [path]
Result: [observed behavior]
- Risk review:
Security:
Data:
Compatibility:
Rollback:

REVIEW BOUNDARY

独立 review 应优先寻找行为回归、安全风险、边界遗漏、缺失测试和与既有模式不一致的实现。审查不是重复摘要，而是寻找可导致事故的具体证据。

09 | 交付协议：让变更可审查、可合并、可回滚

代码写完只是中点
合并后仍要可维护

交付阶段的目标，是把实现结果转化为团队可以审查和维护的变更包。Claude Code 可以协助整理 diff、提交、PR、CI 和发布说明，但最终合并必须建立在人工审查和证据充分的基础上。

COMMIT

描述意图而非文件变化

提交信息应说明变更意图、影响范围和必要的验证线索。避免“update files”或“fix stuff”一类无审查价值的描述。

PULL REQUEST

让 reviewer 快速定位风险

PR 应包含 Summary、Verification、Risk、Rollback 和 reviewer notes。界面或文档改动应给出可检查材料。

CI

把失败输出变成下一轮输入

CI 失败不应只粘贴报错。需要记录失败命令、失败范围、已排除原因和下一步修复策略。

交付门禁

门禁	通过标准	拒绝合并信号
Diff scope	只包含任务相关文件。	混入格式化、临时代码、日志或无关重构。
Verification	最小相关测试通过，高风险变更补充完整验证。	未运行验证且没有合理说明。
Security	无 secret、权限扩大、外部写入或危险默认值。	敏感动作未说明审批和回滚。
Docs	用户或维护者会受影响的行为已更新说明。	代码行为变更但文档仍描述旧流程。
Rollback	能说明撤销提交、关闭开关或回滚迁移的路径。	只能向前修，无法降级或隔离。

10

| 扩展机制：何时使用 MCP、hooks、skills 与 subagents

扩展不是装饰
扩展应解决重复成本或风险

Claude Code 的扩展机制应服务于工程治理，而不是堆叠能力。判断标准很直接：是否减少重复说明，是否缩小上下文污染，是否提升验证可靠性，是否保持权限可审查。

机制	解决的问题	推荐用途	风险控制
MCP	外部系统信息需要反复复制。	issue、PR、监控、设计、只读数据库、内部工具。	只接可信服务器；项目级配置进入审查；凭证不入库。
Hooks	机械守门动作重复出现。	会话启动加载环境、编辑后格式化、提交前 lint、敏感命令拦截。	保持快速、可解释；复杂判断交给 review 或专门工具。
Skills	多步骤流程需要复用。	debug、review、release、migration、文档生成、变更摘要。	技能描述精确；支持文件和脚本可审查。
Subagents	调查工作会污染主上下文。	日志归纳、代码搜索、依赖审计、独立审查、并行研究。	明确任务边界；结果以摘要返回；不要重复主线工作。
Permission settings	需要降低重复确认或限制危险动作。	allowlist 安全命令，denylist 高风险路径。	组织策略优先于个人便利；生产动作保留人工审批。

DESIGN RULE

稳定事实进入记忆层，重复流程进入 skills，机械检查进入 hooks，外部数据进入 MCP，隔离调查进入 subagents。若一个扩展无法说明所属层级，通常还不应被引入团队标准。

11 | 团队治理：从个人效率到组织能力

团队标准不是“每个人怎么写指令”
而是同一套交付门禁

角色分工

角色	责任	不应外包
Tech Lead	定义 workflow、风险门禁、review 标准和团队资产。	架构取舍、合并责任、生产审批。
Engineer	提供任务边界，审查实现，维护 CLAUDE.md 和模板。	业务判断、敏感操作确认。
Reviewer	独立审查 diff、测试证据、安全风险和回滚路径。	以工具摘要替代真实审查。
DevOps / Security	管理权限策略、MCP、hooks、组织级配置和审计。	凭证外泄、生产写入授权。

成熟度模型

阶段	表现	下一步
Individual	个人用 Claude Code 加速小任务。	建立项目级 CLAUDE.md 和验收清单。
Project	项目有共享规则、模板和常用验证命令。	引入 plan/review 门禁和 PR 模板。
Team	团队有统一权限策略、hooks、skills 和 review 规范。	治理 MCP、subagents 和组织级策略。
Organization	配置、审计、培训和安全策略跨项目统一。	建立指标体系：通过率、返工率、风险事件、交付周期。

GOVERNANCE STANDARD

组织层面的目标不是让所有工程师使用相同措辞，而是让同一类任务拥有同一套输入材料、验证命令、风险门禁和交付清单。措辞会变化，协议必须稳定。

12

失败模式：高风险信号与纠偏动作

故障通常不是突然出现
早期信号足够明确

失败模式	早期表现	后果	纠偏动作
Kitchen sink session	一个会话混入多个无关任务。	上下文污染，旧约束干扰新任务。	任务切换时 <code>/clear</code> 或新会话；大型任务用 <code>checklist</code> 管理状态。
Spec gap	只有功能名，没有目标和验收。	实现方向反复变化，diff 扩散。	补写 <code>SPEC.md</code> 与 <code>ACCEPTANCE.md</code> ，计划确认后再实现。
Verification gap	完成摘要没有命令、结果或手动路径。	回归进入 PR 或生产。	要求 <code>evidence ledger</code> ；无法验证则标为未完成。
Overgrown memory	<code>CLAUDE.md</code> 变成项目百科。	上下文成本上升，关键规则被稀释。	删减至稳定规则；路径规则拆入 <code>.claude/rules/</code> ；流程拆入 <code>skills</code> 。
Unsafe automation	<code>hooks</code> 或 MCP 拥有外部写入能力但无审查。	外部系统误写、数据泄露、权限事故。	最小权限、只读优先、项目级配置进入 <code>code review</code> 。
Review theater	<code>review</code> 只复述改动摘要。	真正风险无人检查。	审查必须按行为回归、边界、安全、测试和模式一致性列问题。

STOP RULE

同一问题连续两轮修正仍未收敛，应停止当前路径，清理上下文并以新任务材料重启。

ESCALATION

涉及生产、权限、数据迁移、外部写入或不可逆删除时，自动流程必须升级为人工审批。

RETROSPECTIVE

每次返工都应判断是否需要更新 `CLAUDE.md`、模板、`hook`、`skill` 或团队 `checklist`。

13 | 模板资产与官方来源

手册之外
还应留下可复用文件

随附模板

CLAUDE.md

项目级运行协议，包含命令、架构边界、验证规则和 compact instructions。

SPEC.md

复杂任务的规格模板，用于明确问题、范围、数据接口、验收和风险。

ACCEPTANCE.md

任务验收清单，用于确保目标、验证、review 和 handoff 均有证据。

PR.md

PR 描述模板，帮助 reviewer 快速理解变更、验证和回滚路径。

官方来源

- Claude Code overview: <https://code.claude.com/docs/en/overview>
- Quickstart: <https://code.claude.com/docs/en/quickstart>
- How Claude Code works: <https://code.claude.com/docs/en/how-claude-code-works>
- Common workflows: <https://code.claude.com/docs/en/common-workflows>
- Best practices: <https://code.claude.com/docs/en/best-practices>
- Memory: <https://code.claude.com/docs/en/memory>
- Permission modes: <https://code.claude.com/docs/en/permission-modes>
- Security: <https://code.claude.com/docs/en/security>
- MCP: <https://code.claude.com/docs/en/mcp>
- Hooks: <https://code.claude.com/docs/en/hooks>
- Subagents: <https://code.claude.com/docs/en/sub-agents>
- Skills: <https://code.claude.com/docs/en/skills>

BOUNDARY

本文依据截至 2026-05-04 的官方资料整理。由于 Claude Code 产品形态、权限策略和扩展机制持续变化，团队采用前应复核目标版本、组织策略、MCP 服务器来源、hooks 行为和安全要求。